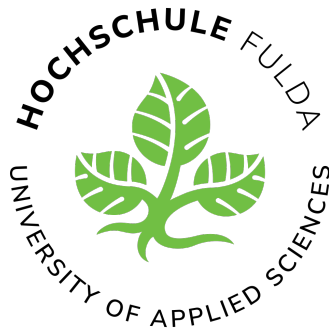




Praxisorientierte Fallstudie:

Nutzwertanalyse von End-to-End-Test-Frameworks (Selenium vs. Cypress) für die Automatisierung repräsentativer regressionskritischer Testszenarien zur Reduzierung des manuellen Testaufwands

Name: Marcel Licht

Studiengang: Bachelor Angewandte Informatik dual

Modul: Aktuelles Thema der Angewandten Informatik – Praxisorientierte Fallstudie

Matrikelnummer: 1540372

Lehrperson: Prof. Dr. Martine Herpers

Unternehmen: JUMO GmbH & Co. KG

Fachbetreuung: Daniel Kircher

Datum: 11. August 2026

Zusammenfassung

Das Testen von Software, insbesondere von regressionskritischen Nutzungsszenarien, ist im Entwicklungsalltag häufig mit einem hohen manuellen und zeitintensiven Aufwand verbunden. Diese Fallstudie untersucht im Kontext der internen Webanwendung „Puma“ der JUMO GmbH & Co. KG, wie dieser Aufwand durch den Einsatz automatisierter End-to-End-Test-Frameworks zielgerichtet reduziert werden kann. Ziel der Arbeit ist es, auf Basis einer strukturierten Evaluierung das am besten geeignete Werkzeug für die spezifischen, stark reglementierten Unternehmensanforderungen zu identifizieren. Hierfür werden die etablierten Frameworks Selenium und Cypress einander gegenübergestellt.

Die methodische Untersuchung kombiniert eine prototypische Implementierung dreier repräsentativer Testszenarien mit einer anschließenden Nutzwertanalyse nach Kühnappel. Die Bewertung erfolgt anhand von fünf betrieblich gewichteten Kriterien: Integration in die Infrastruktur (25 %), Lernkurve und Syntax (20 %), Test-Ausführung und Debugging (20 %), Stabilität (20 %) sowie Dokumentation und Ökosystem (15 %).

Die Ergebnisse der Analyse zeigen, dass Cypress mit einem Gesamtnutzwert von 7,800 aus rein entwicklungstechnischer Sicht dem Framework Selenium (7,200) überlegen ist. Cypress punktet maßgeblich durch eine intuitivere Syntax und signifikant höhere Teststabilität dank integriertem Auto-Waiting. Dennoch offenbart die prototypische Umsetzung gravierende infrastrukturelle Hürden für Cypress: Strenge Proxy-Vorgaben und das etablierte Java-Ökosystem des Unternehmens erschweren eine nahtlose Einbindung erheblich. Selenium lässt sich hingegen als Java-Bibliothek sofort und reibungslos in den bestehenden Maven-Buildprozess integrieren.

Aus diesen Erkenntnissen wird die Handlungsempfehlung abgeleitet, kurzfristig Selenium für die Automatisierung der Tests zu nutzen, um zeitnah erste Automatisierungsgewinne zu erzielen. Mittelfristig sollte das Unternehmen jedoch prüfen, ob die CI/CD-Infrastruktur für Node.js-Umgebungen geöffnet werden kann, um zukünftig von der technischen Überlegenheit von Cypress zu profitieren.

Inhaltsverzeichnis

Abbildungsverzeichnis	5
Tabellenverzeichnis	6
Quellcodeverzeichnis	7
1 Einleitung	8
1.1 Problemstellung und Motivation	8
1.2 Zielsetzung der Arbeit	8
1.3 Forschungsfrage und Aufbau der Arbeit	9
2 Theoretische Grundlagen	10
2.1 Stand der Forschung	10
2.2 Softwaretest und End-to-End-Testing	10
2.3 Betrachtete Test-Frameworks: Selenium und Cypress	10
2.4 Nutzwertanalyse als Bewertungsmethode	11
3 Methodisches Vorgehen	12
3.1 Forschungsdesign der Fallstudie	12
3.2 Auswahl repräsentativer regressionskritischer Testszenarien	12
3.3 Ableitung und Gewichtung der Bewertungskriterien	13
3.4 Vorgehen der prototypischen Umsetzung	13
3.5 Datenerhebung und Auswertung	13
4 Implementierung und Durchführung der Prototypen	14
4.1 Testumgebung und Rahmenbedingungen	14
4.2 Beschreibung der Testszenarien	15
4.2.1 Szenario 1: Login	15
4.2.2 Szenario 2: Neues Projekt bzw. neue Aktivität anlegen	16
4.2.3 Szenario 3: Aktivität bearbeiten	16
4.3 Setup und technische Integration	16
4.3.1 Selenium	17
4.3.2 Cypress	17
4.4 Syntax- und Architekturvergleich	17
4.4.1 Architektur	17
4.4.2 Ausdrucksstärke der Syntax	17
4.4.3 Wartbarkeit	18
4.4.4 Synchronisation	18

4.5	Code-Gegenüberstellung: Projektanlage	18
5	Evaluation und Nutzwertanalyse	21
5.1	Durchführung und Testergebnisse	21
5.2	Auswertung des Kriterienmodells	21
5.2.1	Integration in die Infrastruktur (25 %)	22
5.2.2	Lernkurve und Syntax (20 %)	22
5.2.3	Test-Ausführung und Debugging (20 %)	22
5.2.4	Stabilität (20 %)	22
5.2.5	Dokumentation und Ökosystem (15 %)	22
5.3	Nutzwertanalyse und Gesamtauswertung	22
6	Fazit und Ausblick	24
6.1	Beantwortung der Forschungsfrage und Handlungsempfehlung	24
6.2	Kritische Reflexion und Methodengrenzen	24
6.3	Ausblick	25
	Literaturverzeichnis	26
	Eigenständigkeitserklärung	26
	Anhänge	26
A	Erklärung zur Nutzung von KI-Werkzeugen	26
B	Bewertungsmatrix (Excel)	27
C	TestLog	27

Abbildungsverzeichnis

1	Fachliche und technische Struktur der Webanwendung Puma	15
2	Grafische Gegenüberstellung der Teil- und Gesamtnutzwerte	23
3	Bewertungsmatrix der Nutzwertanalyse (Selenium vs. Cypress) – Teil 1 .	27
4	Bewertungsmatrix der Nutzwertanalyse (Selenium vs. Cypress) – Teil 2 .	28

Tabellenverzeichnis

1	Ergebnis der Nutzwertanalyse	23
2	Ergebnis der Nutzwertanalyse gemäß Bewertungsmatrix	23

Quellcodeverzeichnis

1	Projektanlage mit Selenium	19
2	Projektanlage in Cypress	19

1 Einleitung

1.1 Problemstellung und Motivation

Das Testen von Software ist ein zentraler Bestandteil der Softwareentwicklung. Es stellt sicher, dass Anwendungen den funktionalen Anforderungen entsprechen und zuverlässig arbeiten. Durch Tests lassen sich Fehler frühzeitig erkennen und Probleme vor dem produktiven Einsatz reduzieren.

Softwaretests umfassen verschiedene Testarten und Vorgehensweisen. Während Tests früher häufig manuell durch direkte Interaktion mit der Anwendung durchgeführt wurden, hat sich aufgrund steigender Anforderungen an Qualität und Entwicklungsgeschwindigkeit der Einsatz automatisierter Testverfahren etabliert [1].

Im Kontext der JUMO GmbH & Co. KG wird eine interne Webanwendung (Puma) fortlaufend durch Fehlerbehebungen und funktionale Erweiterungen weiterentwickelt. Trotz der bekannten Vorteile der Automatisierung erfolgen Prüfungen zentraler Nutzungsszenarien derzeit noch manuell, häufig in Form explorativer Tests.

Dieser manuelle Testaufwand ist zeitintensiv, bindet personelle Ressourcen und schränkt die Reproduzierbarkeit der Testergebnisse ein, da die Ausführung stark von der testenden Person abhängt. Daher besteht der betriebliche Bedarf, Ansätze zur Automatisierung regressionskritischer End-to-End-Tests systematisch zu untersuchen. Dies betrifft speziell solche Testszenarien, die nach Änderungen besonders anfällig für Fehler oder unerwünschte Seiteneffekte sind.

Im Fokus dieser Fallstudie stehen die Test-Frameworks Selenium [2] und Cypress [3]. Beide Werkzeuge automatisieren browserbasierte Tests, unterscheiden sich jedoch in ihrer technischen Architektur, der Integration in bestehende Entwicklungsumgebungen, der Wartbarkeit der Testimplementierung sowie den infrastrukturellen Voraussetzungen.

Die Motivation dieser Arbeit ergibt sich aus dem konkreten Bedarf, die Qualitätssicherung bei JUMO im Kontext häufiger Änderungen effizienter zu gestalten. Ziel ist es nicht, ein allgemein überlegenes Framework zu bestimmen, sondern das für diesen spezifischen Unternehmenskontext am besten geeignete Werkzeug durch eine Nutzwertanalyse zu identifizieren.

1.2 Zielsetzung der Arbeit

Ziel der vorliegenden Fallstudie ist es, auf Basis eines strukturierten Vergleichs eine fundierte Entscheidungshilfe für die Auswahl eines End-to-End-Test-Frameworks bei der JUMO GmbH & Co. KG zu erarbeiten.

Dabei ist es nicht das Ziel, ein allgemein überlegenes Framework zu bestimmen, sondern das Werkzeug zu identifizieren, welches für den konkreten Unternehmenskontext und dessen technische Restriktionen am besten geeignet ist. Hierzu werden relevante Anforderungen aus dem Betrieb abgeleitet, ausgewählte Nutzungsszenarien der „Puma“-

Webanwendung prototypisch in beiden Frameworks umgesetzt und die Ergebnisse anhand einer Nutzwertanalyse systematisch bewertet.

1.3 Forschungsfrage und Aufbau der Arbeit

Aus der formulierten Zielsetzung leitet sich die zentrale Forschungsfrage dieser Fallstudie ab:

Welches der beiden Frameworks (Selenium oder Cypress) ist für die Automatisierung ausgewählter repräsentativer regressionskritischer End-to-End-Testszenarien im gegebenen Unternehmenskontext besser geeignet, wenn Integrationsaufwand, Wartbarkeit, Stabilität, Ausführungseffizienz und Unterstützungsangebote systematisch berücksichtigt werden?

Zur präzisen Beantwortung der Hauptfrage werden folgende Teilfragen untersucht:

- Welche Anforderungen ergeben sich aus dem betrieblichen Umfeld an ein End-to-End-Test-Framework?
- Welche Nutzungsszenarien der Webanwendung eignen sich als repräsentative Testfälle für einen Framework-Vergleich?
- Wie unterscheiden sich Selenium und Cypress hinsichtlich des technischen und organisatorischen Integrationsaufwands?
- Welche Unterschiede zeigen sich in Bezug auf Verständlichkeit, Wartbarkeit und Änderbarkeit des Testcodes?
- Wie unterscheiden sich die Frameworks bezüglich Stabilität, Laufzeit und Diagnosemöglichkeiten im Fehlerfall?

Aufbau der Arbeit:

Im Anschluss an diese Einleitung werden in Kapitel 2 die theoretischen Grundlagen des End-to-End-Testings sowie der untersuchten Frameworks Selenium und Cypress erläutert. Kapitel 3 widmet sich der Methodik und beschreibt das Vorgehen der Nutzwertanalyse inklusive der Entwicklung des Kriterienmodells. In Kapitel 4 erfolgt die Beschreibung der prototypischen Umsetzung und der Datenerhebung anhand der ausgewählten Testszenarien. Kapitel 5 präsentiert die Auswertung der Ergebnisse mittels der Nutzwertanalyse. Die Arbeit schließt in Kapitel 6 mit einer Diskussion der Erkenntnisse und einem zusammenfassenden Fazit.

2 Theoretische Grundlagen

Dieses Kapitel legt das theoretische Fundament für die Fallstudie. Es beleuchtet zunächst den aktuellen Stand der Forschung, definiert die Rolle von End-to-End-Tests in der Qualitätssicherung, stellt anschließend die beiden zu vergleichenden Frameworks vor und erläutert abschließend die Methodik der Nutzwertanalyse.

2.1 Stand der Forschung

Die Evaluierung und der Vergleich von Testautomatisierungswerkzeugen sind Gegenstand zahlreicher Publikationen der angewandten Informatik. Während grundlegende Werke wie Liggesmeyer [4] oder Spillner und Linz [1] die theoretischen Konzepte des Softwaretests und der Testautomatisierung umfassend behandeln, fokussieren sich neuere Publikationen häufig auf den Architekturvergleich spezifischer Frameworks. Der Kontrast zwischen traditionellen WebDriver-basierten Ansätzen wie Selenium und modernen, browserinternen Werkzeugen wie Cypress wird in der Fachliteratur oft anhand von Metriken wie Ausführungsgeschwindigkeit, Stabilität (Flakiness) und API-Design diskutiert.

Eine Lücke besteht jedoch oft in der detaillierten Betrachtung dieser Tools innerhalb stark reglementierter Unternehmensnetzwerke und spezifischer Legacy-Infrastrukturen (wie z. B. ältere JDK-Versionen oder strikte Proxy-Vorgaben). Die vorliegende Fallstudie knüpft an den bestehenden Forschungsstand an, erweitert ihn jedoch um eine systematische Bewertung, die exakt diese praxisnahen infrastrukturellen und organisatorischen Rahmenbedingungen der JUMO GmbH & Co. KG in den Mittelpunkt stellt.

2.2 Softwaretest und End-to-End-Testing

Die Testautomatisierung ist ein fest etablierter Bestandteil moderner Softwarequalitätssicherung [4]. Während Unit- und Integrationstests isolierte Code-Abschnitte oder Schnittstellen auf technischer Ebene prüfen, testen End-to-End-Tests (E2E-Tests) das gesamte System über alle Schichten hinweg – von der Benutzeroberfläche bis zur Datenbank. Sie dienen insbesondere dazu, regressionskritische Nutzungsszenarien aus Sicht der Endanwender in einer realitätsnahen Umgebung zu überprüfen und dadurch zentrale Geschäftsprozesse abzusichern [1]. Da E2E-Tests in der Regel komplexer in der Einrichtung und laufzeitintensiver sind, ist die Wahl eines robusten, wartbaren und in die Systemlandschaft passenden Automatisierungs-Frameworks entscheidend für die langfristige Akzeptanz im Entwicklungsalltag.

2.3 Betrachtete Test-Frameworks: Selenium und Cypress

Für den praktischen Vergleich dieser Arbeit stehen die Werkzeuge Selenium und Cypress im Mittelpunkt.

Selenium ist ein langjährig etabliertes Standardwerkzeug der browserbasierten Testautomatisierung. Es steuert den Browser über das native WebDriver-Protokoll an und bietet breite Integrationsmöglichkeiten in unterschiedliche Programmiersprachen, wie

beispielsweise Java [2]. Diese hohe Flexibilität und Rückwärtskompatibilität erfordern jedoch im Gegenzug oftmals einen höheren manuellen Konfigurationsaufwand, insbesondere bei der Handhabung von asynchronen Ladezeiten und der Synchronisation des Testablaufs.

Cypress wird demgegenüber häufig als moderne, entwicklerzentrierte Alternative für Webanwendungen herangezogen. Im Gegensatz zu Selenium läuft Cypress direkt im Browser und führt Tests in derselben Event-Loop wie die Anwendung selbst aus [3]. Dies ermöglicht eine direkte Rückmeldung während der Testausführung, ein automatisches Warten auf DOM-Elemente (*Auto-Waiting*) sowie sehr gute Debugging-Möglichkeiten. Da Cypress jedoch primär auf JavaScript beziehungsweise TypeScript ausgelegt ist, bedarf es bei der Integration in bestehende Java-Ökosysteme oder stark reglementierte Firmennetzwerke einer gesonderten Betrachtung.

Der Mehrwert der vorliegenden Arbeit liegt demnach nicht in einer rein allgemeinen Toolbeschreibung, sondern in der kontextbezogenen Bewertung dieser Frameworks für konkrete Webanwendungen unter realen infrastrukturellen Restriktionen im Unternehmensumfeld.

2.4 Nutzwertanalyse als Bewertungsmethode

Zur strukturierten und objektiven Entscheidungsfindung wird die Nutzwertanalyse verwendet. Diese Methode erlaubt es, mehrere Handlungsalternativen anhand systematisch gewichteter qualitativer und quantitativer Kriterien nachvollziehbar zu vergleichen [5]. Das Verfahren gliedert sich in vier zentrale Schritte: die Definition der relevanten Bewertungskriterien, deren Gewichtung auf Basis der betrieblichen Anforderungen, die Ermittlung der Zielerfüllung (Punktevergabe) durch prototypische Messungen sowie die finale Berechnung des Gesamtnutzwertes. Damit wird ein bestehender praktischer Bedarf mit einer methodisch fundierten Bewertungslogik verbunden, um eine kontextbezogene Entscheidungshilfe abzuleiten.

3 Methodisches Vorgehen

Um die Eignung der Frameworks Selenium und Cypress für den Einsatz bei der JUMO GmbH & Co. KG systematisch und objektiv zu vergleichen, wurde ein praxisorientiertes Forschungsdesign gewählt. Dieses Kapitel beschreibt den methodischen Aufbau der Fallstudie, die Auswahl der Testszenarien, die Definition der Bewertungskriterien sowie den Ablauf der praktischen Erprobung.

3.1 Forschungsdesign der Fallstudie

Die Untersuchung basiert auf einer Kombination aus prototypischer Implementierung und einer strukturierten Entscheidungsfindung mittels Nutzwertanalyse (NWA) nach Kühnapfel [5]. Da ein rein theoretischer Vergleich der Frameworks den spezifischen infrastrukturellen Kontext des Unternehmens nicht ausreichend berücksichtigen würde, werden beide Werkzeuge in der realen Entwicklungsumgebung erprobt.

Die Ergebnisse dieser praktischen Umsetzung fließen in die Nutzwertanalyse ein. Diese Methode erlaubt es, qualitative und quantitative Beobachtungen in messbare Punktwerte zu übersetzen, diese mit unternehmensspezifischen Gewichtungen zu versehen und so zu einem nachvollziehbaren Gesamtergebnis zu gelangen.

3.2 Auswahl repräsentativer regressionskritischer Testszenarien

Da eine vollständige Automatisierung aller Anwendungsfälle der „Puma“-Webanwendung den Rahmen dieser Studie sprengen würde, wurden repräsentative und regressionskritische Nutzungsszenarien ausgewählt. Diese Szenarien bilden die Kernfunktionalitäten der Anwendung ab und sind nach funktionalen Erweiterungen besonders fehleranfällig.

Für die prototypische Umsetzung wurden folgende drei Szenarien definiert:

1. **Authentifizierung:** Ein Login-Vorgang zur Prüfung des Zugriffs auf geschützte Bereiche. Obwohl dieser Bereich bei funktionalen Erweiterungen der Anwendung selten direkt modifiziert wird, ist er aufgrund seiner hohen Kritikalität (Totalausfall bei Defekt) zwingend abzusichern. Zudem dient er im Framework-Vergleich als methodische Baseline zur Evaluation grundlegender Interaktions- und Wartemechanismen.
2. **Formularverarbeitung:** Das Ausfüllen, Validieren und Absenden eines typischen Eingabeformulars (Projekt-/Aktivitätsanlage). Dieses Szenario vergleicht, wie die Frameworks mit verschiedenen dynamischen UI-Elementen (z. B. Select2-Dropdowns) umgehen.
3. **Datensatzbearbeitung:** Ein mehrstufiger Prozess, bei dem ein Datensatz aufgerufen, modifiziert, gespeichert und verifiziert wird. Dies testet primär, wie zuverlässig die Werkzeuge asynchrone Ladezeiten und die zusammenhängende Navigation handhaben.

3.3 Ableitung und Gewichtung der Bewertungskriterien

Die Bewertungskriterien für die Nutzwertanalyse wurden direkt aus den fachlichen, technischen und organisatorischen Anforderungen des Ausbildungsbetriebs abgeleitet. Die Gewichtung (in Klammern) erfolgt vor dem Hintergrund der Unternehmensziele:

- **Integration in die Infrastruktur (25 %):** Bewertung der initialen Einrichtung. Ein Test-Framework im betrieblichen Umfeld der JUMO GmbH & Co. KG kann nur effizient skaliert werden, wenn es die strengen Proxy-Vorgaben und das etablierte Java-Ökosystem respektiert. Ein zu hoher administrativer Overhead negiert den Automatisierungsgewinn, weshalb dieses Kriterium am höchsten gewichtet wird.
- **Lernkurve und Syntax (20 %):** Beurteilung, wie intuitiv die Erstellung und Pflege von Tests ist, um eine schnelle Einarbeitung neuer Entwickler zu gewährleisten.
- **Test-Ausführung und Debugging (20 %):** Analyse der Ausführungsgeschwindigkeit und der Möglichkeiten zur Fehlersuche.
- **Stabilität (20 %):** Zuverlässigkeit der Tests (Minimierung von *Flaky Tests*). Laufzeit und Stabilität bestimmen direkt den Erfolg der täglichen Qualitätssicherung.
- **Dokumentation und Ökosystem (15 %):** Bewertung der offiziellen Hilfen und Community. Dies flankiert den Entwicklungsprozess, ist den technischen Kerna-
spekten jedoch untergeordnet.

3.4 Vorgehen der prototypischen Umsetzung

Die praktische Erprobung erfolgt schrittweise. Zunächst wird für beide Frameworks eine lokale Testumgebung aufgesetzt, wobei notwendige Konfigurationsschritte und Workarounds dokumentiert werden. Anschließend werden die drei definierten Testszenarien nacheinander in Selenium und Cypress implementiert. Um eine Vergleichbarkeit zu gewährleisten, wird für beide Werkzeuge derselbe Ausgangszustand der Anwendung genutzt.

3.5 Datenerhebung und Auswertung

Die Datenerhebung findet parallel zur prototypischen Umsetzung statt. Quantitative Daten (Dauer der Testausführung, Code-Struktur) und qualitative Daten (Hürden bei der Netzwerkintegration, Fehlermeldungen) werden strukturiert festgehalten.

Nach Abschluss der Implementierung werden die erhobenen Daten in das Kriterienmodell überführt. Jedes Framework erhält pro Kriterium eine Bewertung auf einer festen **Skala von 1 bis 10 Punkten**. Diese Punktzahl wird mit der zuvor definierten Gewichtung multipliziert. Die Summe aller gewichteten Punkte ergibt den Gesamtnutzwert.

4 Implementierung und Durchführung der Prototypen

Dieses Kapitel stellt die praktische Testumgebung, die ausgeführten Testszenarien sowie den technischen und syntaktischen Vergleich zwischen Selenium und Cypress dar. Die Ausformulierung stellt dabei konkrete Projektbezüge zur Webanwendung Puma bei der JUMO GmbH & Co. KG her.

4.1 Testumgebung und Rahmenbedingungen

Die Tests wurden im betrieblichen Kontext der JUMO GmbH & Co. KG durchgeführt. Untersuchungsgegenstand ist die interne Webanwendung *Puma*, die aus einem konkreten betrieblichen Bedarf heraus entstanden ist. Da die vorherige Verwaltung firmeninterner Projekte zunehmend unübersichtlich und schwer nachvollziehbar wurde, dient Puma nun als zentrale Lösung zur Projekt- und Kostenverwaltung. Ein Projekt entspricht in der Anwendung im Wesentlichen einem Arbeitsplan, der um eine detaillierte Kosten- und Stundenplanung erweitert wurde. Darüber hinaus bietet das System Funktionen gängiger Projektplanungstools: Es lassen sich Personen und Teams zuordnen, relevante Dokumente zentral hinterlegen und Projektdaten über eine Schnittstelle direkt mit dem SAP-System synchronisieren.

Das in Abbildung 1 dargestellte vereinfachte Komponentendiagramm veranschaulicht die fachliche und technische Struktur der Anwendung. Im Mittelpunkt steht die Projektverwaltung als zentrale Entität. Ein Projekt ist mit Modulen für Kosten- und Stundenplanungen, Ressourcen, Teams sowie der Dokumentenverwaltung verknüpft. Zur Sicherstellung der Nachvollziehbarkeit von Änderungen werden Aktivitäten, Kommentare, Benachrichtigungen und Historieneinträge erfasst. Die Suchfunktionen werden über Apache Solr abgebildet. Technische Basisdienste wie Logging, Caching, Fehlerbehandlung und allgemeine Hilfsklassen unterstützen die fachlichen Komponenten im Hintergrund.

Der Technologie-Stack der Anwendung basiert auf Java. Als Entwicklungsumgebung kam Eclipse zum Einsatz, während Maven für das Build-Management verwendet wurde. Die Bereitstellung der Anwendung für die lokale Testausführung erfolgte über einen lokalen Tomcat-Server. Der verwendete Port für die Testausführung ist **8082**. Ohne diesen laufenden Tomcat-Server ist keine Testausführung möglich, da die Webanwendung lokal ansonsten nicht erreichbar ist.

Die Ausführung der Tests unterlag den strengen Sicherheitsrichtlinien des Firmennetzwerks. Aufgrund der unternehmensinternen Proxy-Vorgaben war die Installation externer Komponenten (wie beispielsweise npm-Pakete für Node.js) nur eingeschränkt möglich.

Die Tests wurden lokal auf einem Entwickler-Laptop durchgeführt. Die Spezifikationen des Systems lauten wie folgt:

- **Betriebssystem:** Windows 11 Enterprise
- **Version:** 24H2

- **Installationsdatum:** 29.08.2025
- **Betriebssystembuild:** 26100.8457

Für die Testdurchführung wurde ausschließlich der Browser Microsoft Edge in der Version 120 verwendet.

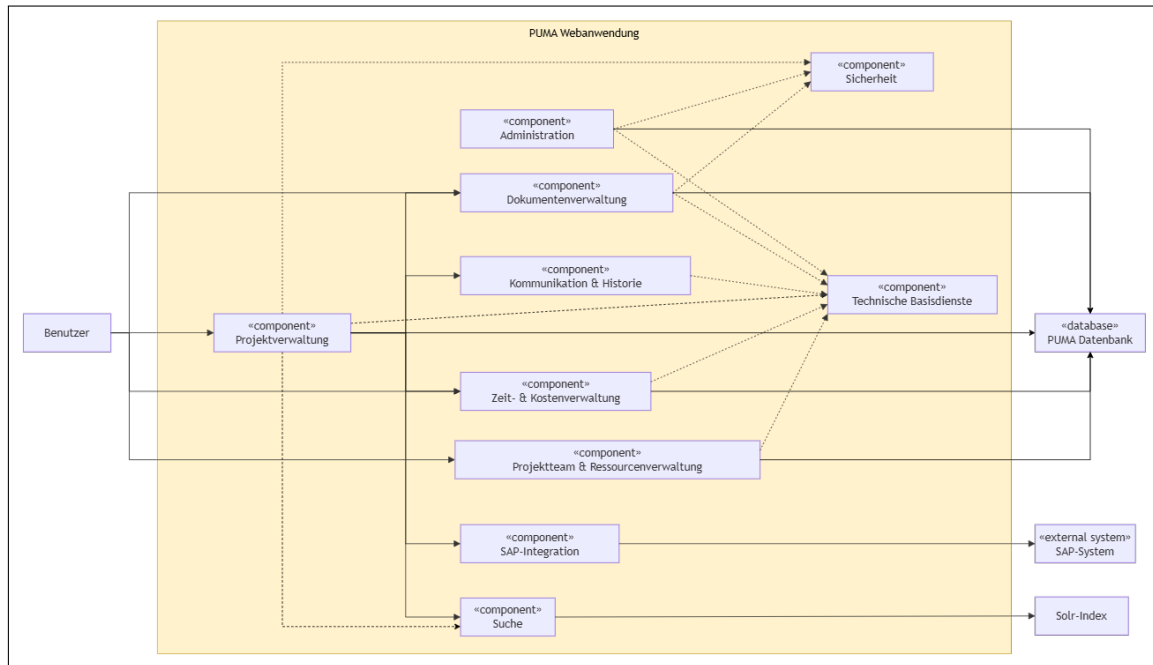


Abbildung 1: Fachliche und technische Struktur der Webanwendung Puma

4.2 Beschreibung der Testszenarien

Dieses Unterkapitel beschreibt die getesteten Anwendungsszenarien systematisch und ordnet diese den jeweiligen Testimplementierungen zu. Zur besseren Übersichtlichkeit wird im Folgenden auf die Angabe vollständiger Dateipfade verzichtet. Stattdessen werden die zugehörigen Testdateien direkt mit ihrem Dateinamen benannt. Die Namenskonvention orientiert sich an einer fortlaufenden Nummerierung der Szenarien (z. B. 01_ . . . für das erste Szenario). Die vollständigen Pfade und Quellcodes sind im digitalen Anhang (Git-Repository) zu finden.

4.2.1 Szenario 1: Login

Das Ziel dieses Tests ist es zu prüfen, ob sich ein Benutzer erfolgreich an der Puma-Webanwendung anmelden kann. Die Startbedingungen setzen voraus, dass Puma lokal über den Tomcat-Server auf Port 8082 erreichbar ist und ein gültiger Benutzer mit Benutzernamen und Passwort zur Verfügung steht.

Der Ablauf des Tests beginnt mit dem Öffnen der Login-Seite. Nach der Eingabe von Benutzernamen und Passwort wird der Login-Button betätigt. Anschließend wird geprüft, ob eine Weiterleitung auf die erwartete Zielseite erfolgt.

Das erwartete Ergebnis ist, dass der Benutzer erfolgreich angemeldet wird und der Redirect im Test validiert wird. Die automatisierte Umsetzung erfolgte in den Testdateien **01_PumaLoginTest.java** (Selenium) und **01_login-ui.cy.js** (Cypress).

4.2.2 Szenario 2: Neues Projekt bzw. neue Aktivität anlegen

In diesem Szenario wird geprüft, ob ein neues Projekt bzw. eine neue Aktivität korrekt über die Benutzeroberfläche angelegt werden kann. Als Startbedingung muss der Benutzer erfolgreich angemeldet sein. Die Anwendung muss lokal erreichbar sein und die benötigten Dropdown-Werte müssen im System zur Verfügung stehen.

Der Test navigiert zur Maske für die Projekt- bzw. Aktivitätsanlage. Das Stammbblatt wird durch Eingabe der benötigten Projektdaten ausgefüllt. Eine Besonderheit bilden hier Dropdowns mit **select2**, die dynamisch reagieren und daher gezielt im Code angesprochen werden müssen. Nach dem Ausfüllen wird das Formular gespeichert und die Systemrückmeldung geprüft.

Als erwartetes Ergebnis wird das Projekt oder die Aktivität erfolgreich angelegt. Die Anwendung zeigt eine korrekte Bestätigung an, und die Daten sind im System vorhanden. Die zugehörigen Testdateien sind **02_NewProjektTest.java** und **02_NewActivityAnlegenTest.java** für Selenium sowie **02_newProjekt.cy.js** und **02_newActivity.cy.js** für Cypress.

4.2.3 Szenario 3: Aktivität bearbeiten

Das Ziel des dritten Szenarios ist die Prüfung, ob eine bestehende Aktivität geöffnet, geändert, gespeichert und anschließend korrekt validiert werden kann. Die Startbedingungen erfordern, dass der Benutzer angemeldet ist und eine bestehende Aktivität aufgerufen werden kann.

Der Ablauf sieht die Navigation zur Aktivität sowie das Öffnen der Bearbeitungsansicht vor. Es folgt die Änderung definierter Werte (z. B. Fortschritt oder Datum). Nach dem Speichern wird die Aktivität erneut aufgerufen, um zu validieren, ob die Änderungen korrekt übernommen wurden.

Es wird erwartet, dass die Aktivität erfolgreich gespeichert wird und die geänderten Werte korrekt in der Oberfläche angezeigt werden. Die Umsetzung für dieses Szenario befindet sich in **03_EditActivityTest.java** und **03_ControlEditedActivityTest.java** (Selenium) sowie **03_editActivity.cy.js** und **03_controlEditedActivity.cy.js** (Cypress).

4.3 Setup und technische Integration

Dieses Unterkapitel erläutert, wie Selenium und Cypress technisch in die bestehende Entwicklungsumgebung integriert wurden und welche spezifischen Herausforderungen dabei aufgetreten sind.

4.3.1 Selenium

Selenium wurde als Bibliothek in die bestehende Java-Umgebung integriert. Die Einbindung erfolgte unkompliziert über die **pom.xml** mit Maven. Da im Projekt ohnehin mit Java, Eclipse und Maven gearbeitet wird, passt das Framework sehr gut zur bestehenden Entwicklungsumgebung. Die Integration verlief weitgehend reibungslos. Ein anfängliches Kompatibilitätsproblem mit der im Unternehmen verwendeten älteren JDK-Version konnte umgangen werden.

Selenium ließ sich aufgrund der bestehenden Java- und Maven-Struktur gut in die vorhandene Entwicklungsumgebung integrieren.

4.3.2 Cypress

Im Gegensatz zur etablierten Java-basierten Integration von Selenium setzt Cypress eine Node.js- und npm-Umgebung voraus. Im vorliegenden Unternehmenskontext erwies sich diese technologische Anforderung als erste Hürde: Strikte Proxy- und Sicherheitsrichtlinien blockierten die reguläre Paketinstallation über npm, weshalb eine dedizierte IT-Freigabe zwingend erforderlich war.

Zwar kann eine lokale Installation über ein ZIP-Archiv als kurzfristiger Workaround dienen, dieser Ansatz wurde jedoch verworfen. Er eignet sich nicht für wiederholbare, skalierbare Continuous-Integration-Prozesse (CI) und erschwert die saubere Versionsverwaltung. Folglich bleibt der initiale Einrichtungsaufwand für Cypress – trotz der grundsätzlichen Lösbarkeit durch administrative Freigabeprozesse – im direkten Vergleich zu Selenium organisatorisch und technisch umständlicher.

4.4 Syntax- und Architekturvergleich

Die Frameworks Selenium und Cypress unterscheiden sich grundlegend in Bezug auf Architektur, Syntax, Wartbarkeit und Synchronisation.

4.4.1 Architektur

Selenium wird als Bibliothek in die bestehende Java/JUnit-Umgebung integriert [1]. Es arbeitet als Bestandteil des vorhandenen Maven-Projekts, wodurch die Testautomatisierung eng mit der bestehenden Backend- bzw. Java-Struktur verbunden ist. Cypress hingegen ist ein eigenständiges Tooling auf Basis von Node.js und bringt eine eigene Testumgebung mit. Es läuft nicht als Java-Bibliothek, sondern agiert als separates End-to-End-Testframework.

Zusammenfassend: Selenium erweitert die vorhandene Java-Testumgebung, während Cypress eine eigenständige Testplattform auf Basis von Node.js bereitstellt.

4.4.2 Ausdrucksstärke der Syntax

Cypress zeichnet sich durch eine Syntax aus, die näher an natürlicher Sprache ist. Befehle lassen sich über das Konzept des Chainings gut lesbar aneinanderreihen. Durch Befehle

wie **cy.get(...).type(...)** entsteht eine kompakte und gut lesbare Testbeschreibung.

Selenium ist hingegen technischer und stärker objektorientiert geprägt. Bevor eine Interaktion stattfinden kann, müssen DOM-Elemente in der Regel explizit als **WebElement** gespeichert werden. Die tatsächliche Interaktion erfolgt erst im nächsten Schritt über Methoden wie **clear()** und **sendKeys()**.

4.4.3 Wartbarkeit

Bei der Nutzung von Selenium entstehen ohne strukturierende Maßnahmen schnell Wiederholungen im Quellcode. Um Redundanzen zu vermeiden, sind Hilfsmethoden oder die Anwendung des Page-Object-Patterns als notwendige Strukturierungsmaßnahmen unerlässlich [4].

Cypress ermöglicht durch Chaining und implizite Mechanismen häufig kompakteren Code. Gerade einfache Testabläufe wirken dadurch übersichtlicher, auch wenn bei steigender Projektkomplexität ebenso in Cypress eine strukturierte Testorganisation erforderlich sein kann.

4.4.4 Synchronisation

Ein zentraler Unterschied liegt in der Handhabung dynamischer Ladezeiten. In Selenium müssen dynamische Elemente häufig explizit mit Anweisungen wie **wait.until(...)** behandelt werden. Die Sichtbarkeit oder Klickbarkeit muss aktiv geprüft werden, was den Code umfangreicher macht.

Cypress verfügt über integrierte Auto-Waits. Viele Wartebedingungen sind bereits in den Basisbefehlen wie **cy.get()**, **type()** oder **click()** enthalten, wodurch sich der manuelle Synchronisationsaufwand maßgeblich verringert.

Der wesentliche Unterschied liegt darin, dass Selenium Wartebedingungen häufig explizit formuliert, während Cypress viele Synchronisationsaufgaben automatisch übernimmt.

4.5 Code-Gegenüberstellung: Projektanlage

Anhand eines ausgewählten Codebeispiels wird im Folgenden der strukturelle Aufbau beider Frameworks bei typischen UI-Interaktionen gegenübergestellt. Der Schwerpunkt dieses Vergleichs liegt auf dem Befüllen von Formularfeldern, konkret dem Setzen des Projektnamens und des Enddatums bei der Projektanlage.

Wie in Listing 1 erkennbar, wird für jedes relevante Element in Selenium zunächst ein explizites Warten (Wait) definiert. Beim Feld `project_name` wird auf die Sichtbarkeit gewartet, beim Feld `project_endDate` auf die Klickbarkeit. Erst danach können die Felder geleert und mit den Testdaten befüllt werden. Dies macht den Code zwar robust gegenüber dynamischen Ladezeiten, erhöht jedoch die Zeilenanzahl und verleiht dem Test einen stark technischen Charakter.

```

1 // Projektname setzen + prüfen
2 WebElement projektname = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id(
3     "project_name")));
4 assertNotNull(projektname, "Feld 'Projektname' wurde nicht gefunden.");
5 projektname.clear();
6 projektname.sendKeys(PROJEKTNAME);
7 assertEquals(PROJEKTNAME, projektname.getAttribute("value"),
8     "Projektname wurde nicht korrekt gesetzt.");
9 // Bereich
10 selectDropdownByTextAndAssert("project_divisionId", "Informationstechnologie (IT)");
11
12 // Projektprofil
13 selectDropdownByTextAndAssert("project_sapProjectProfile", "ZEW");
14
15 // Profitcenter
16 selectDropdownByTextAndAssert("project_sapProfitCenter", "CC10100400 - Ausbildung");
17
18 // Enddatum
19 WebElement endDate = wait.until(ExpectedConditions.elementToBeClickable(By.id("
20     project_endDate")));
21 assertNotNull(endDate, "Feld 'Enddatum' wurde nicht gefunden.");
22 endDate.clear();
23 endDate.sendKeys("17.08.2026");
24 assertEquals("17.08.2026", endDate.getAttribute("value"),
25     "Enddatum wurde nicht korrekt gesetzt.");

```

Quellcode 1: Projektanlage mit Selenium

```

1 // Neues Projekt anlegen + Aktivität
2 // Stammbblatt ausfüllen (Pflichtfelder):
3
4 // Projektname
5 cy.get('#project_name').type('Cypress Testprojekt')
6
7 // Bereich
8 cy.get('#project_divisionId')
9     .scrollIntoView()
10     .select('Informationstechnologie (IT)', { force: true })
11
12 // Projektprofil
13 cy.get('#project_sapProjectProfile')
14     .select('ZEW', { force: true })
15
16 // Profitcenter
17 cy.get('#project_sapProfitCenter')
18     .select('CC10100400 - Ausbildung', { force: true })
19
20 // Datum(Zeitspanne)
21 //Von
22 /*
23 cy.get('#project_startDate')
24     .clear()
25     .type('dd.mm.yyyy')
26 */
27
28 //Bis
29 cy.get('#project_endDate')
30     .scrollIntoView()
31     .clear()
32     .type('17.08.2026')

```

Quellcode 2: Projektanlage in Cypress

Cypress nutzt im Gegensatz dazu, wie in Listing 2 dargestellt, die Anweisung `cy.get()` zur Auswahl des Elements. Notwendige Wartebedingungen sind in den Cypress-Standardbefehlen bereits implizit integriert, sodass die Aktionen direkt verkettet (Chaining) werden können. Für das Datumsfeld wird zusätzlich `scrollIntoView()` verwendet, um sicherzustellen, dass sich das Element im sichtbaren Viewport befindet.

Aus dieser Gegenüberstellung lassen sich folgende Bewertungsaspekte für die Nutzwertanalyse ableiten:

1. **Synchronisation:** Selenium erfordert bei dynamischen Oberflächen die manuelle Definition expliziter Waits. Cypress kapselt diese Wartebedingungen in seinen Standardbefehlen.
2. **Codeumfang:** Selenium benötigt mehr Programmzeilen, da Wartebedingungen und Elementvariablen separiert werden. Cypress erreicht durch das Chaining eine kompaktere Darstellung.
3. **Lesbarkeit:** Der Selenium-Code ist stark technisch strukturiert. Die Cypress-Syntax liest sich hingegen eher wie eine direkte funktionale Beschreibung der Anwenderaktion.
4. **Wartbarkeit:** Während Cypress einfache UI-Abläufe nativ kompakt hält, erfordert Selenium bei wachsendem Testumfang zwingend den Einsatz von Hilfsmethoden oder Architekturmustern (wie dem Page Object Model), um die Wartbarkeit sicherzustellen.

Zusammenfassend zeigt der Vergleich, dass Selenium eine sehr feingranulare Steuerung der Synchronisation ermöglicht, was jedoch den Code-Overhead erhöht. Cypress bietet insbesondere bei formularbasierten Interaktionen eine lesbarere und kompaktere Syntax, was die Einarbeitung und Pflege der Tests erleichtert.

5 Evaluation und Nutzwertanalyse

Nachdem im vorherigen Kapitel die prototypische Umsetzung der Testszenarien beschrieben wurde, erfolgt in diesem Kapitel die systematische Auswertung. Ziel ist es, die erhobenen Daten zu strukturieren und eine Basis für die Beantwortung der Forschungsfrage zu schaffen.

5.1 Durchführung und Testergebnisse

Im Rahmen der praktischen Evaluation wurden die ausgewählten regressionskritischen Testszenarien iterativ ausgeführt, um Daten hinsichtlich der Ausführungsgeschwindigkeit und der Zuverlässigkeit (Stabilität) zu erheben.

Die Ausführungsgeschwindigkeiten beider Frameworks zeigten beim initialen Testlauf identische Werte von jeweils circa 45 Sekunden (inklusive Start der Testumgebung und Browser-Initialisierung). Bei den darauffolgenden Folgeläufen reduzierte sich die Ausführungszeit bei beiden Tools auf eine Spanne von 22 bis 32 Sekunden. Cypress schließt beim vollständigen Projekttest minimal schneller ab, was darauf zurückzuführen ist, dass Aktionen direkter im Browser ausgeführt werden, während Selenium über den WebDriver mit expliziten Wartebedingungen arbeitet.

Zur Bewertung der *Flakiness* (Unzuverlässigkeit) wurden die Szenarien für den *Login* sowie das *Ändern von Datensätzen* auf 50 Durchläufe skaliert. Die Tendenz war signifikant: Bei Cypress traten in 4 von 50 Durchläufen Probleme auf, während Selenium mit 11 von 50 Durchläufen eine deutlich höhere Fehlerquote verzeichnete. Selenium reagiert wesentlich empfindlicher auf Ladezeiten dynamischer Ajax-Elemente, wohingegen Cypress durch integrierte Retry-Mechanismen robuster arbeitet.

5.2 Auswertung des Kriterienmodells

Die strukturierte Auswertung basiert auf der Bewertungsmatrix nach Kühnapfel [5]. Das Modell umfasst fünf Hauptkriterien. Für jedes Unterkriterium i wird eine Bewertung B_i auf der in Kapitel 3 definierten Skala von 1 bis 10 vergeben. Diese Teilbewertung wird mit der prozentualen Gewichtung g_i multipliziert, um den Kategorienwert K_j zu ermitteln:

$$K_j = \sum_{i=1}^n (B_i \cdot g_i) \quad (1)$$

Der Gesamtnutzwert N ergibt sich schließlich aus der Summe aller gewichteten Hauptkategorien:

$$N = \sum_{j=1}^m K_j \quad (2)$$

5.2.1 Integration in die Infrastruktur (25 %)

Selenium (8,0 Punkte, gewichtet: 2,000): Die bestehende Umgebung ist stark auf Java, Eclipse und Maven ausgerichtet. Selenium ließ sich reibungslos über die `pom.xml` integrieren. **Cypress (5,0 Punkte**, gewichtet: 1,250): Das Framework erfordert eine Node.js-Umgebung. In der Praxis blockierten firmeninterne Proxys den `npm install`-Prozess, was administrative IT-Freigaben zwingend erforderlich machte.

5.2.2 Lernkurve und Syntax (20 %)

Cypress (9,0 Punkte, gewichtet: 1,800) überzeugt durch eine intuitive, an der manuellen Bedienung orientierte Syntax (`cy.visit()`, `cy.get().click()`). **Selenium (7,0 Punkte**, gewichtet: 1,400) erfordert ein höheres technisches Verständnis der Java-Programmierung und explizite Selektor-Instanziierungen, was den initialen Overhead erhöht.

5.2.3 Test-Ausführung und Debugging (20 %)

Cypress (8,0 Punkte, gewichtet: 1,600) bietet mit dem modernen grafischen Test Runner und der *Time-Travel*-Funktion exzellente Diagnosemöglichkeiten. **Selenium (7,0 Punkte**, gewichtet: 1,400) punktet im Gegenzug durch das klassische IDE-Debugging via JUnit-Breakpoints in Eclipse, liefert aber weniger visuelles Feedback.

5.2.4 Stabilität (20 %)

Cypress (9,0 Punkte, gewichtet: 1,800) erzeugt, wie in Kapitel 5.1 belegt, durch natives Auto-Waiting weitaus stabilere UI-Tests. **Selenium (6,0 Punkte**, gewichtet: 1,200) erfordert bei asynchronen Oberflächen stetige Eingriffe durch `wait.until(...)`. Fehlen diese, entstehen *flaky tests*.

5.2.5 Dokumentation und Ökosystem (15 %)

Cypress (9,0 Punkte, gewichtet: 1,350) besticht durch praxisorientierte Web-Dokumentationen. **Selenium (8,0 Punkte**, gewichtet: 1,200) liefert eine riesige Community und Best-Practices, wirkt jedoch teils generischer.

5.3 Nutzwertanalyse und Gesamtauswertung

Tabelle 2 visualisiert die Ergebnisse der Verrechnung von Punktwerten und Gewichtungen, welche methodisch in Kapitel 3.3 hergeleitet wurden.

Das Fazit der Nutzwertberechnung zeigt ein klares Bild: Cypress ist mit einem Gesamtnutzwert von 7,995 aus rein entwicklungstechnischer Sicht (fachlich, syntaktisch und hinsichtlich der Teststabilität) dem Framework Selenium (6,905) überlegen. Einer sofortigen und reibungslosen Einführung im JUMO-Netzwerk werden jedoch aktuell durch die infrastrukturellen Rahmenbedingungen gravierende Hürden gesetzt.

Tabelle 1: Ergebnis der Nutzwertanalyse

Kriterium	Gewichtung	Selenium (gew.)	Cypress (gew.)
Integration in die Infrastruktur	25 %	2,000	1,250
Lernkurve und Syntax	20 %	1,400	1,800
Test-Ausführung und Debugging	20 %	1,400	1,600
Stabilität	20 %	1,200	1,800
Dokumentation und Ökosystem	15 %	1,200	1,350
Gesamtnutzwert	100 %	7,200	7,800

Tabelle 2: Ergebnis der Nutzwertanalyse gemäß Bewertungsmatrix

Kriterium	Gewichtung	Selenium (gew.)	Cypress (gew.)
Integration in die Infrastruktur	25 %	1,775	1,700
Lernkurve und Syntax	20 %	1,330	1,750
Test-Ausführung und Debugging	20 %	1,400	1,520
Stabilität	20 %	1,200	1,720
Dokumentation und Ökosystem	15 %	1,200	1,305
Gesamtnutzwert	100 %	6,905	7,995

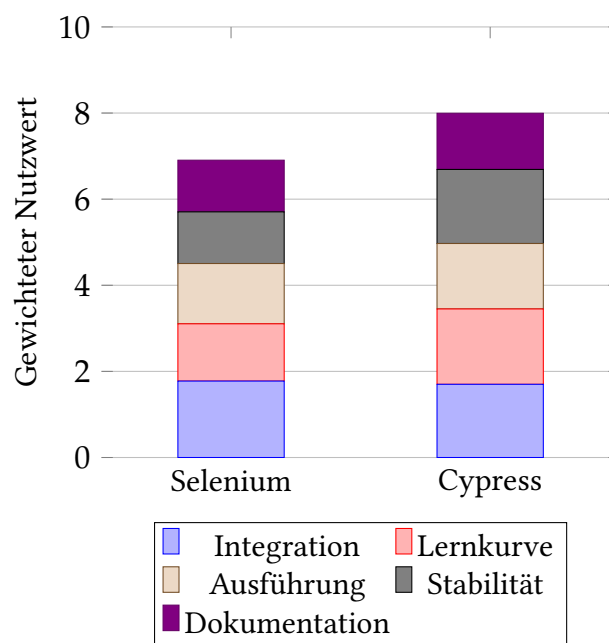


Abbildung 2: Grafische Gegenüberstellung der Teil- und Gesamtnutzwerte

6 Fazit und Ausblick

6.1 Beantwortung der Forschungsfrage und Handlungsempfehlung

Ziel dieser Fallstudie war es zu evaluieren, welches End-to-End-Test-Framework – Selenium oder Cypress – sich besser für die Automatisierung regressionskritischer Testszenarien im Umfeld der JUMO GmbH & Co. KG eignet. Die durchgeführte Nutzwertanalyse liefert hierauf eine differenzierte Antwort.

Aus rein technischer und entwicklungsorientierter Perspektive erweist sich **Cypress** als das überlegene Tool. Es punktet durch eine intuitive Syntax, exzellente Debugging-Möglichkeiten und eine signifikant höhere Stabilität bei dynamischen Webanwendungen. Die integrierten Retry-Mechanismen reduzieren die Fehleranfälligkeit bei Ladezeiten drastisch.

Dennoch führt diese Überlegenheit nicht zu einer uneingeschränkten Empfehlung. Die infrastrukturellen Voraussetzungen der JUMO GmbH & Co. KG basieren stark auf einem Java-Ökosystem, während Cypress eine Node.js-Umgebung erfordert. Die in der Evaluation aufgetretenen Konflikte mit den strengen Proxy- und Sicherheitsrichtlinien des Unternehmens bremsen den Automatisierungsgewinn derzeit aus.

Selenium hingegen ist durch seine Java-Kompatibilität direkt und ohne administrativen Zusatzaufwand in die bestehende Infrastruktur integrierbar. Es erfordert zwar einen höheren Programmieraufwand zur Sicherstellung der Teststabilität, stellt aber unter den aktuellen Rahmenbedingungen die pragmatischere Lösung dar.

Die **Handlungsempfehlung** für das Unternehmen lautet daher: Kurzfristig sollte Selenium für die Automatisierung der regressionskritischen Tests der Puma-Anwendung genutzt werden, da die Integration in das bestehende Java-/Maven-Umfeld sofort möglich ist. Mittelfristig sollte jedoch geprüft werden, ob die IT-Infrastruktur so angepasst werden kann (z. B. durch Freigaben im Proxy und die Bereitstellung von Node.js in der CI/CD-Pipeline), dass ein Wechsel auf Cypress möglich wird, um von dessen überlegener Teststabilität und Entwicklerfreundlichkeit zu profitieren.

6.2 Kritische Reflexion und Methodengrenzen

Die in dieser Fallstudie angewandte Methodik und die daraus abgeleiteten Ergebnisse unterliegen gewissen Einschränkungen, die bei der abschließenden Bewertung zu berücksichtigen sind.

Zunächst basiert die Bewertung der CI- und Reporting-Integration für beide Tools teilweise auf theoretischen Annahmen. Im Rahmen des zeitlich begrenzten Praxisprojekts lag der Fokus auf der prototypischen Erstellung lokaler Tests; eine vollständige Integration in eine automatisierte Build-Pipeline wurde nicht abschließend praktisch umgesetzt. Des Weiteren ist zu beachten, dass qualitative Bewertungskriterien wie *Lesbarkeit der Syntax* und *Usability des Test Runners* zwangsläufig einer subjektiven Wahrnehmung des

Evaluierenden unterliegen, auch wenn sie durch konkrete Implementierungserfahrungen untermauert wurden.

Abschließend ist eine klare fachliche Differenzierung bezüglich der identifizierten Schwächen von Cypress notwendig: Die gravierendsten Punktabzüge, welche bei der Projekterstellung und Installation auftraten, stellen keine originären Schwächen des Tools dar. Sie resultieren vielmehr maßgeblich aus den rigiden Sicherheits- und Proxy-Richtlinien der Unternehmensinfrastruktur. Würde das Unternehmen standardmäßig Node.js-Umgebungen und entsprechende Paket-Freigaben unterstützen, fiel die Nutzwertanalyse noch deutlicher zugunsten von Cypress aus.

6.3 Ausblick

Zukünftige Arbeiten könnten die in dieser Fallstudie identifizierten Einschränkungen aufgreifen. Eine logische Fortführung wäre die praktische Implementierung beider Frameworks in einer vollständigen CI/CD-Pipeline (z. B. Jenkins oder GitLab CI), um die theoretischen Annahmen zur Reporting- und Ausführungsfähigkeit im automatisierten Build-Prozess empirisch zu belegen.

Darüber hinaus sollte das Unternehmen die strategische Entscheidung treffen, inwiefern moderne JavaScript-/TypeScript-basierte Testing-Tools zukünftig gefördert werden sollen, um langfristig die Effizienz in der Qualitätssicherung von Webanwendungen zu steigern.

Literaturverzeichnis

- [1] A. Spillner und T. Linz, *Basiswissen Softwaretest: Aus-und Weiterbildung zum Certified Tester–Foundation Level nach ISTQB-Standard*. dpunkt. verlag, 2024.
- [2] Selenium Project, *Selenium Documentation*, <https://www.selenium.dev/documentation/>, [Online; abgerufen am 13.05.2026], 2024.
- [3] Cypress.io, *Cypress Documentation*, <https://docs.cypress.io/>, [Online; abgerufen am 13.05.2026], 2024.
- [4] P. Liggesmeyer, *Software-Qualität: Testen, Analysieren und Verifizieren von Software*, 2. Spektrum Akademischer Verlag, 2009.
- [5] J. B. Kühnapfel, *Scoring und Nutzwertanalysen: Ein Leitfaden für die Praxis*. Springer, 2021.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Alle sinngemäß und wörtlich übernommenen Textstellen aus fremden Quellen wurden kenntlich gemacht.

Fulda, den 17.08.2026

Marcel Licht

A Erklärung zur Nutzung von KI-Werkzeugen

Bei der Erstellung dieses Exposés wurden KI-Werkzeuge unterstützend genutzt:

Verwendete Tools: Google Gemini 3.1 Pro

- Hilfe bei der sprachlichen Überarbeitung und Verbesserung eigener Texte (Formulierungshilfe, Grammatik, und Rechtschreibprüfung)
- Hilfe bei LaTeX Befehlen und Formatierungen (z.B. Quellen in der references.bib oder Tabelle)
- Überprüfung auf Vollständigkeit der Anforderungen
- Unterstützung bei der Literaturrecherche

B Bewertungsmatrix (Excel)

In diesem Abschnitt ist die vollständige Bewertungsmatrix der Nutzwertanalyse abgebildet, die als Grundlage für die finale Auswertung diente. Aufgrund des Umfangs der Kriterien und Begründungen wurde die Darstellung der Excel-Datei in zwei Abbildungen aufgeteilt.

Kriterium	Beschreibung	Gewichtung	Selenium (Score 1-10)	Cypress (Score 1-10)
Integration (Java/ Eclipse)	Bewertet wird, wie nahtlos das Tool in unsere Java-/Eclipse-Umgebung integrierbar ist: Setup in Eclipse, Debugging, Build-Integration (Maven/Gradle), Test-Runner (z. B. JUnit/TestNG), CI-Ausführung, Reporting und Pflegeaufwand. Hoher Score = wenig Zusatztools/Workarounds, stabiler Standard-Workflow in IDE und CI.	25%	1,775	1,7
Projekt-Anlage & Tool-Start in deinem Workflow	Wie einfach ist das Setup zum Testen zu integrieren	40%	8	5
Debugging & Ausführen	Wie gut kannst du Tests starten/stoppen, Fehler nachvollziehen	30%	6	9
CI/Reporting Anschlussfähigkeit	Wie gut lässt es sich später in Pipeline + Reports integrieren	30%	7	7
Lenkbarkeit & Syntax	Bewertet wird, wie schnell das Team produktiv wird und wie gut lesbar/wartbar die Syntax ist: Verständlichkeit der API, Boilerplate-Anteil, Konsistenz, typische Patterns (z. B. Page Objects), Qualität der Fehlermeldungen und wie leicht neue Teammitglieder gute Tests schreiben können. Hoher Score = schnelle Einarbeitung + klare, wartbare Test-Codestruktur.	20%	1,33	1,75
Einstieg / erster Test	Wie schnell verstehst du die Basics und bekommst einen Test hin?	40%	7	9
Usability	Wie intuitiv findet man sich ein?	10%	6	9
Lesbarkeit & Wartbarkeit	Kannst du den Test nach 2 Wochen noch leicht verstehen/ändern?	25%	7	8
Fehlermeldungen / Diagnose	Wie klar sind Errors? Wie schnell findest du die Ursache?	25%	6	9
Test-Ausführung	Bewertet wird die Geschwindigkeit im Alltag: Start-/Setup-Zeit, Laufzeit einzelner Tests und Suites, Parallelisierung, Ressourcenverbrauch (CPU/RAM) sowie Skalierung in CI. Hoher Score = schnelle Feedback-Zyklen (lokal & CI) und effiziente Parallel-Ausführung ohne übermäßigen Infrastrukturbedarf.	20%	1,4	1,52
Geschwindigkeit lokal	Wie schnell laufen Tests beim Entwickeln?	60%	7	8
Geschwindigkeit in CI / Skalierung	Bleibt es schnell, wenn es mehr Tests werden / parallel läuft?	40%	7	7
Stabilität	Bewertet wird die Zuverlässigkeit der Testergebnisse: wie gut das Tool Timing/Asynchronität abfängt (Waits/Auto-Waiting), Robustheit gegen UI-Änderungen, Stabilität in CI, sowie Diagnosefähigkeit (Logs/Traces/Screenshots) zur schnellen Fehleranalyse. Hoher Score = reproduzierbare Ergebnisse und niedrige Flaky-Rate.	20%	1,2	1,72
Warten & Synchronisation	Verhalten mit dynamischen Seiten klar, ohne viele Sleeps	40%	6	8
Zuverlässigkeit über mehrere Läufe	Wie oft "random" Fail ohne Codeänderung	55%	6	9
Debug-Hilfen bei Fehlern	Screenshots/Video/Trace/Logs (hilfreich?)	5%	6	9
Dokumentation & Community	Bewertet wird die Unterstützung durch Doku und Ökosystem: Qualität/Aktualität der Dokumentation, Umfang an Beispielen, Community-Aktivität, Problemlösung (Issues/Foren), Release-/Wartungsniveau und verfügbare Integrationen/Plugins. Hoher Score = schnell Lösungen finden und langfristig gut wartbar.	15%	1,2	1,305
Doku-Qualität	Verständlich? Gibt es aktuell, gute Beispiele	70%	8	9
Hilfe (Community)	Schnelle Lösung bei Stagnation	10%	8	8
Ökosystem/Tools	Reporter, Plugins, Integrationen	20%	8	8
Summe (Gesamtnutzwert)			6,905	7,995

Abbildung 3: Bewertungsmatrix der Nutzwertanalyse (Selenium vs. Cypress) – Teil 1

C TestLog

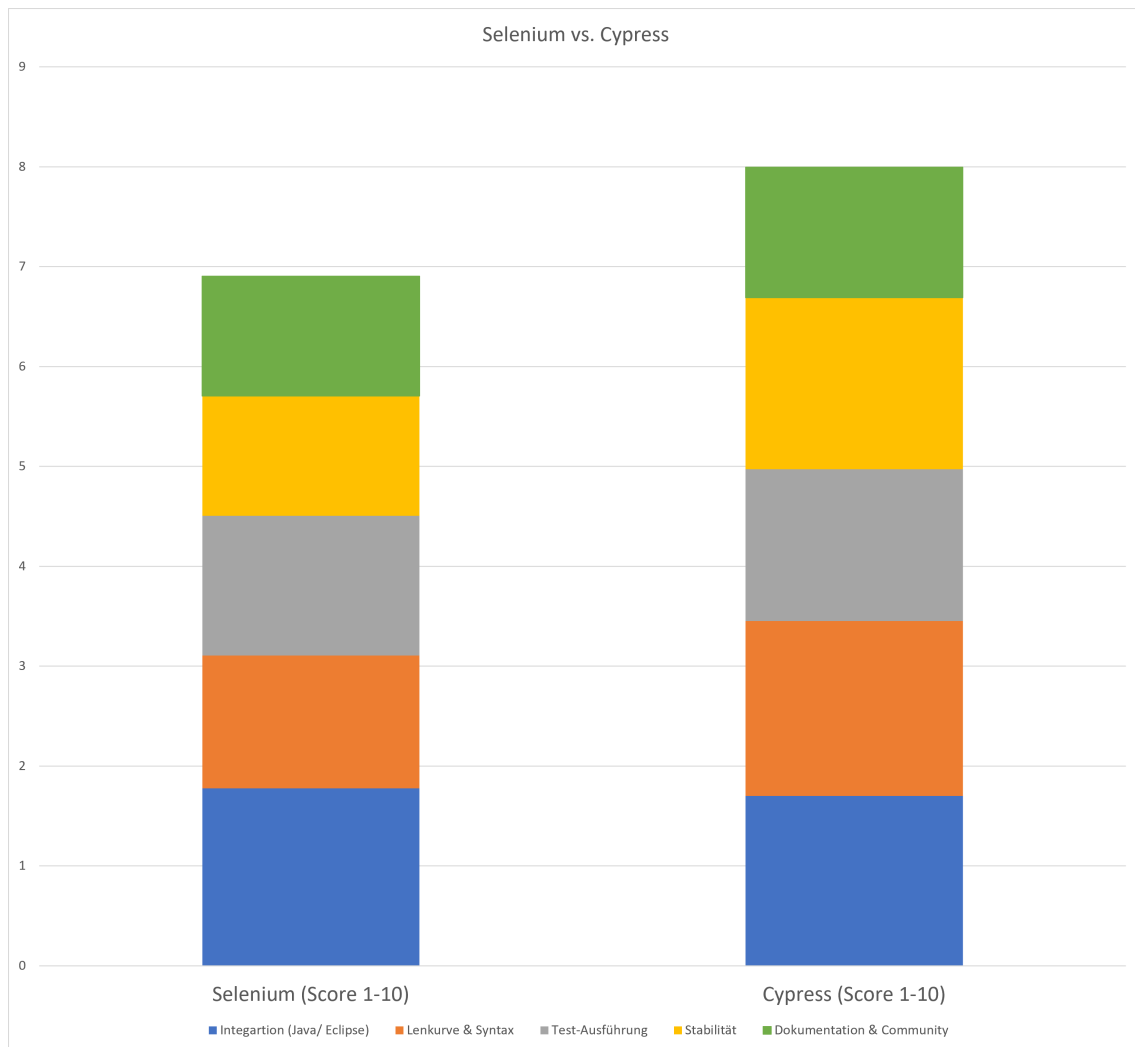


Abbildung 4: Bewertungsmatrix der Nutzwertanalyse (Selenium vs. Cypress) – Teil 2